SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	N. Lahari	**Reg. No.:**	192473018
Date:	22-10-2026	**Duration:**	60 minutes

Code of Conduct

I ___N. Lahari (192473018)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __N. Lahari_____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

1. INTRODUCTION

Artificial Intelligence (AI) is increasingly used in customer support to answer questions, provide product information, resolve common complaints, and assist customers at any time. AI-powered customer support systems can reduce the workload of customer service employees and provide faster responses to routine queries.

However, AI systems may sometimes generate incorrect answers, misunderstand ambiguous questions, or provide unsuitable responses to privacy-sensitive and high-impact requests. If these situations are handled entirely by AI without human supervision, they may reduce customer trust and create operational risks.

The AI-Assisted Customer Support Platform is designed to automate routine customer interactions while identifying situations that require human intervention. The platform maintains conversation context, classifies customer requests, detects uncertainty, integrates with secure external APIs, and escalates sensitive or complex cases to human support staff.

The system also maintains secure audit records and monitors response quality, performance, reliability, and fairness. Therefore, it combines the speed of AI automation with the judgment and responsibility of human support staff.

2. PROBLEM DESCRIPTION

Organizations receive a large number of customer requests related to account information, product details, order status, billing, complaints, and technical support. Handling every request manually requires significant time, effort, and operational cost.

Although AI assistants can answer routine questions, they may fail when a request is unclear, complicated, sensitive, or outside their knowledge. An automated response may also be inappropriate when the customer requests a decision that requires human judgment.

The main problem is to design and develop an AI-assisted customer support platform that can automatically handle routine requests and identify cases that require human intervention.

The platform must classify incoming requests, understand the conversation context, estimate uncertainty, identify sensitive cases, and transfer conversations to the appropriate human agent when necessary. It must

also protect customer information, maintain audit records, support secure API communication, and operate reliably under different workloads.

The system should provide a balance between automation and human oversight so that customer service becomes faster, safer, more reliable, and more transparent.

3. OBJECTIVES

The major objectives of the AI-Assisted Customer Support Platform are:

1. **Automate routine customer requests:** Develop an AI assistant that can answer frequently asked questions, provide basic product information, and handle common customer service requests without requiring a human agent for every interaction.
2. **Detect uncertain conversations:** Identify questions that are ambiguous, incomplete, outside the system's knowledge, or likely to receive an unreliable response.
3. **Escalate sensitive cases:** Transfer complaints, privacy-related requests, high-impact decisions, and complex problems to qualified human support staff whenever human judgment is required.
4. **Maintain conversation context:** Preserve relevant information from previous messages so that the assistant and human agent can understand the customer's issue without repeatedly asking for the same details.
5. **Protect customer information:** Use appropriate authentication, authorization, encryption, and access controls to protect customer conversations and personal information.
6. **Provide secure API integration:** Connect the platform with approved systems such as order management, customer relationship management, and ticket management services through secure APIs.
7. **Monitor service quality:** Track response accuracy, escalation rates, response time, system availability, and customer feedback to identify areas for improvement.
8. **Support responsible AI:** Promote privacy, fairness, transparency, accountability, and human supervision throughout the customer support process.

4. SCOPE OF THE PROJECT

The proposed platform can be used by organizations that provide customer support through websites, mobile applications, or other digital communication channels.

The system handles routine requests automatically, identifies complex or sensitive situations, and transfers such cases to human agents. It maintains the conversation history and relevant case details so that support staff can continue the interaction effectively.

The platform also supports external API integration, secure data management, performance monitoring, testing, and scalable deployment. It can be developed using modular services and containerized components so that different parts of the application can be maintained and scaled independently.

The scope includes AI-assisted conversation handling, request classification, uncertainty detection, escalation management, audit logging, and service monitoring. Final decisions on sensitive or high-impact cases remain under appropriate human control.

5. EXISTING SYSTEM AND ITS LIMITATIONS

In a traditional customer support system, customers contact support employees through telephone calls, email, chat, or support tickets. Employees read the requests, collect relevant information, investigate the problem, and provide solutions.

Some organizations use basic chatbots that answer predefined questions using fixed rules and responses. These systems can handle simple and repetitive queries but may struggle with complex conversations or questions that are not included in their predefined knowledge base.

The major limitations of existing systems are:

1. **High workload:** Human support employees spend considerable time answering repeated questions, which reduces the time available for complicated customer problems.
2. **Slow response:** Customers may need to wait for an available employee, especially during busy periods or outside normal working hours.
3. **Limited understanding:** Rule-based chatbots may fail to understand ambiguous language, follow-up questions, or conversations that require context from previous messages.
4. **Risk of incorrect responses:** AI assistants may produce inaccurate or unsuitable answers when they lack sufficient information or misunderstand the customer's request.
5. **Insufficient escalation control:** A basic chatbot may not reliably identify when a conversation requires human judgment or specialist support.

6. **Privacy and security concerns:** Inadequately protected systems may expose customer information or allow unauthorized access to conversation records.
7. **Limited monitoring:** Without proper logs and performance measurements, organizations may find it difficult to identify repeated failures and improve service quality.

These limitations demonstrate the need for an intelligent customer support platform that combines automation with uncertainty detection, secure information handling, and controlled human escalation.

6. PROPOSED SYSTEM

The proposed system is an AI-Assisted Customer Support Platform that processes customer requests, generates suitable responses, and determines whether each conversation can be handled automatically or should be transferred to a human agent.

When a customer submits a message, the system first identifies the request type and retrieves relevant conversation context. The AI assistant then determines whether sufficient information is available to answer the question.

For routine and low-risk requests, the system generates a response using approved information and authorized services. If the request is unclear, the AI is uncertain, or the issue involves sensitive information or a high-impact decision, the system initiates a human escalation process.

During escalation, the platform creates or updates a support ticket and transfers the conversation history, request category, reason for escalation, and relevant case information to an authorized support agent. The agent can then investigate the problem and provide an appropriate response.

The system records important actions in secure audit logs and continuously monitors service performance. This helps the organization evaluate the quality of AI responses and improve the platform over time.

The proposed solution therefore reduces repetitive work, improves response speed, strengthens human oversight, and supports responsible AI-based customer service.

7. FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations that the system must perform.

Requirement	Description
Customer interaction	Accept customer messages through a supported website, application, or chat interface.
Request classification	Identify the type of request, such as product information, order status, billing, or complaint.
Context management	Maintain relevant previous messages and conversation details during an interaction.
AI response generation	Generate relevant responses for routine requests using approved information.
Uncertainty detection	Identify unclear requests, missing information, and responses that may be unreliable.
Risk identification	Recognize privacy-sensitive, high-impact, or otherwise restricted requests.
Human escalation	Transfer selected conversations to the appropriate human support team.
Ticket management	Create or update support tickets and preserve the reason for escalation.
API integration	Communicate securely with authorized external services when required.
Audit logging	Record important system actions, escalation decisions, and service events.
Performance monitoring	Track response times, system errors, escalation frequency, and service quality.

8. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality, security, and operational characteristics of the platform.

1. **Performance:** The platform should process routine customer requests with minimal delay and provide timely responses during normal operating conditions.
2. **Scalability:** The system should support increasing numbers of customers and conversations by allowing additional service instances to be deployed when necessary.
3. **Reliability:** The platform should handle failures appropriately, recover from temporary service interruptions, and avoid losing important conversation or ticket information.
4. **Security:** Customer information must be protected through authentication, authorization, encryption, secure API communication, and suitable access restrictions.

5. **Privacy:** The system should collect and retain only the information required for its purpose and follow applicable privacy requirements.
6. **Maintainability:** A modular design should allow individual components to be updated, tested, and maintained without unnecessarily affecting the entire application.
7. **Usability:** Customers and support agents should be able to use the platform through clear and understandable interfaces.
8. **Transparency:** Customers should be informed when they are interacting with an AI assistant, and escalation decisions should have understandable reasons.
9. **Fairness:** The system should be evaluated for inconsistent or unfair outcomes across different types of customers and requests.
10. **Auditability:** Important system activities should be recorded securely so that authorized personnel can investigate incidents and review system decisions.

9. SYSTEM ACTORS AND USE CASES

The main actors involved in the platform are the customer, AI assistant, human support agent, administrator, and external service systems.

Customer: The customer submits questions, provides necessary details, receives answers, and requests additional assistance when required.

AI Assistant: The AI assistant classifies requests, uses conversation context, generates routine responses, detects uncertainty, and recommends escalation when necessary.

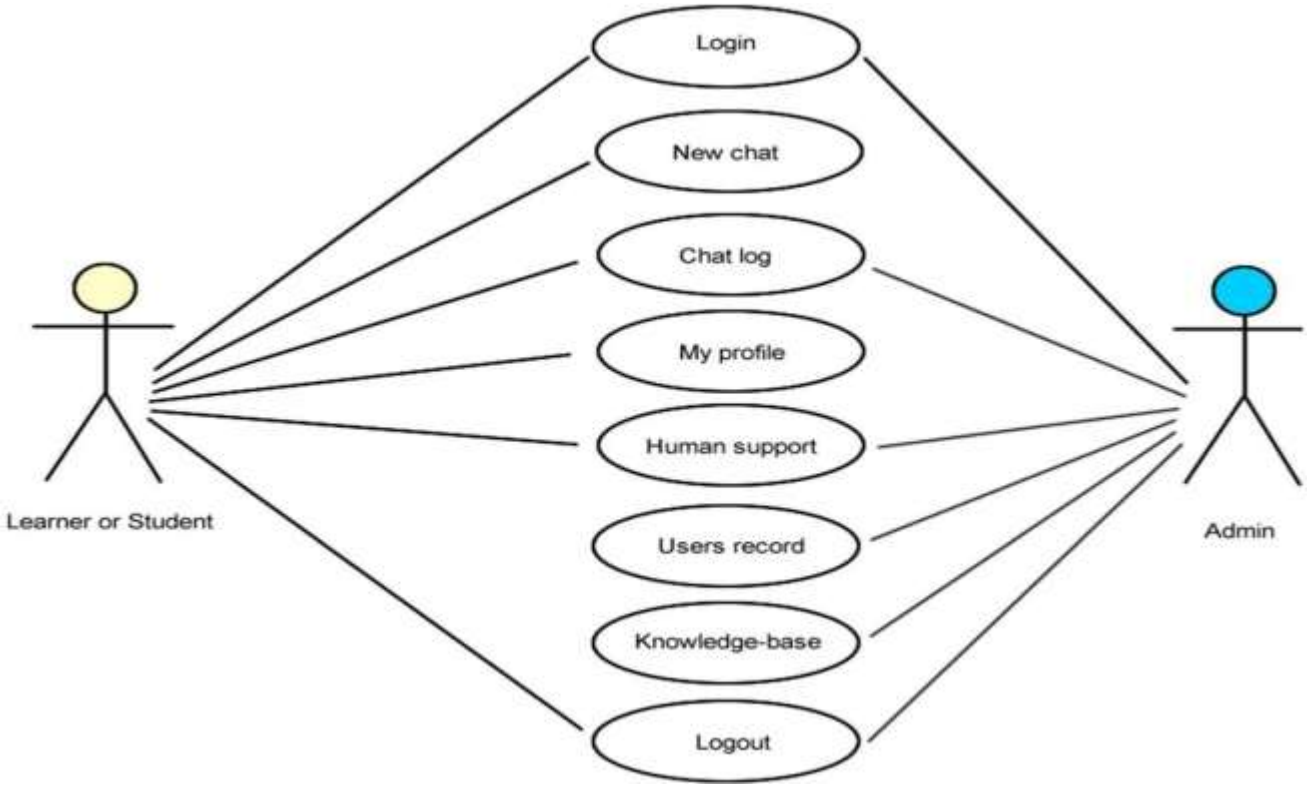
Human Support Agent: The agent receives escalated conversations, reviews the available information, investigates the issue, and provides a suitable response or resolution.

Administrator: The administrator manages authorized users, system configurations, escalation rules, access permissions, and monitoring settings.

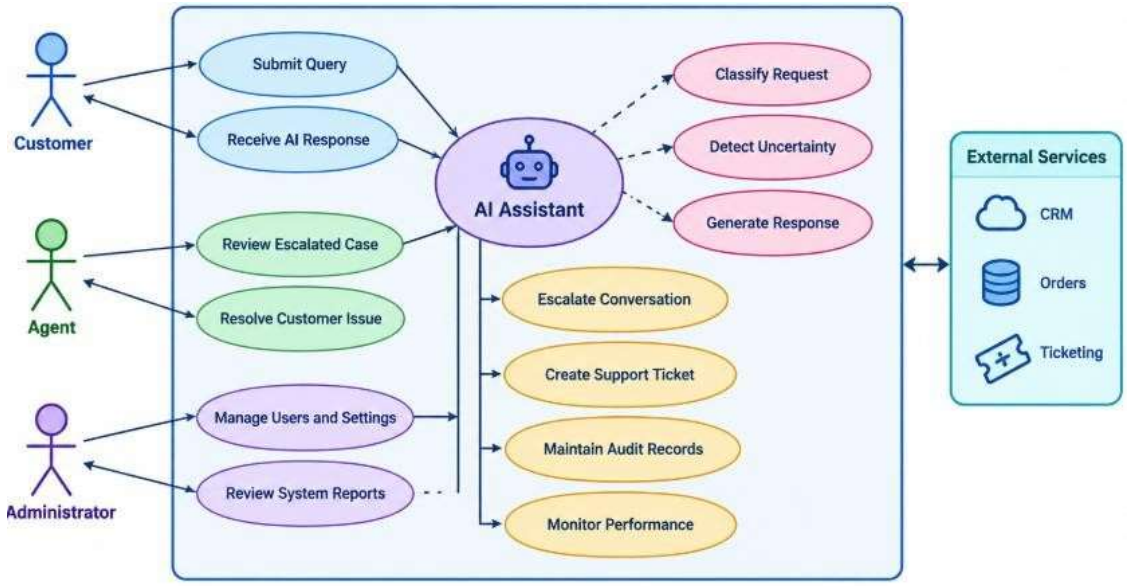
External Services: These services provide approved information such as order status, customer account details, or support ticket information through secure APIs.

The main use cases include submitting a customer query, receiving an AI-generated response, classifying a request, detecting uncertainty, escalating a conversation, assigning a support ticket, reviewing audit records, and monitoring system performance.

USE-CASE DIAGRAM



The use-case diagram should show the relationship between the customer, AI assistant, human support agent, administrator, and external services. Include use cases such as Submit Query, Receive Response, Classify Request, Detect Uncertainty, Escalate Conversation, Manage Ticket, Review Audit Logs, and Monitor Performance.



11. SYSTEM ARCHITECTURE

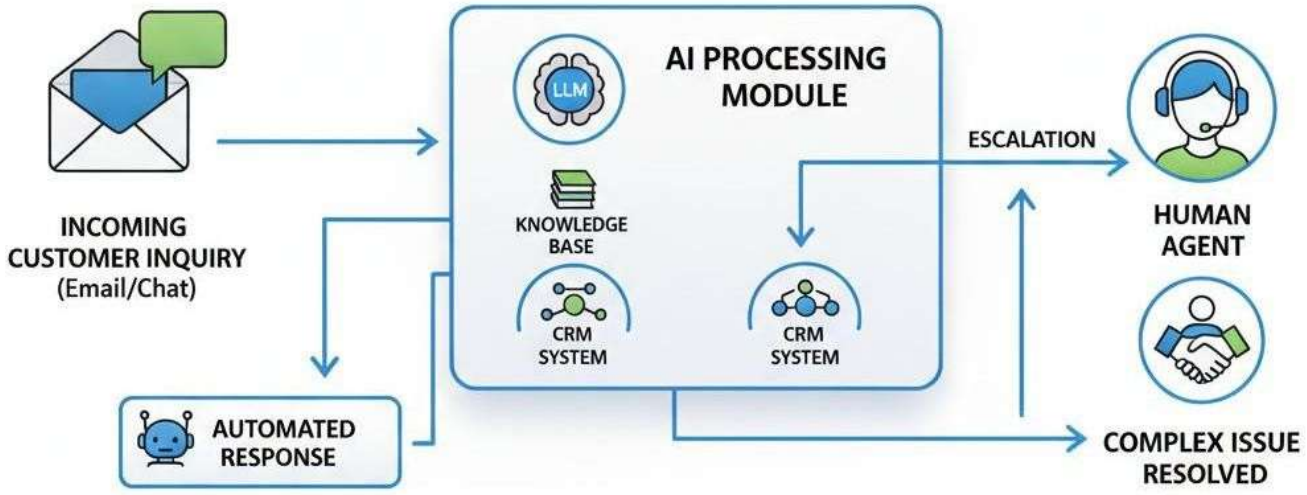
The platform follows a modular architecture in which different components perform specific responsibilities. This improves maintainability, scalability, testing, and fault isolation.

The customer interface is the entry point for all requests. The API gateway or backend entry layer validates incoming requests and directs them to the appropriate services. The conversation manager maintains relevant context, while the request classifier identifies the purpose of the message.

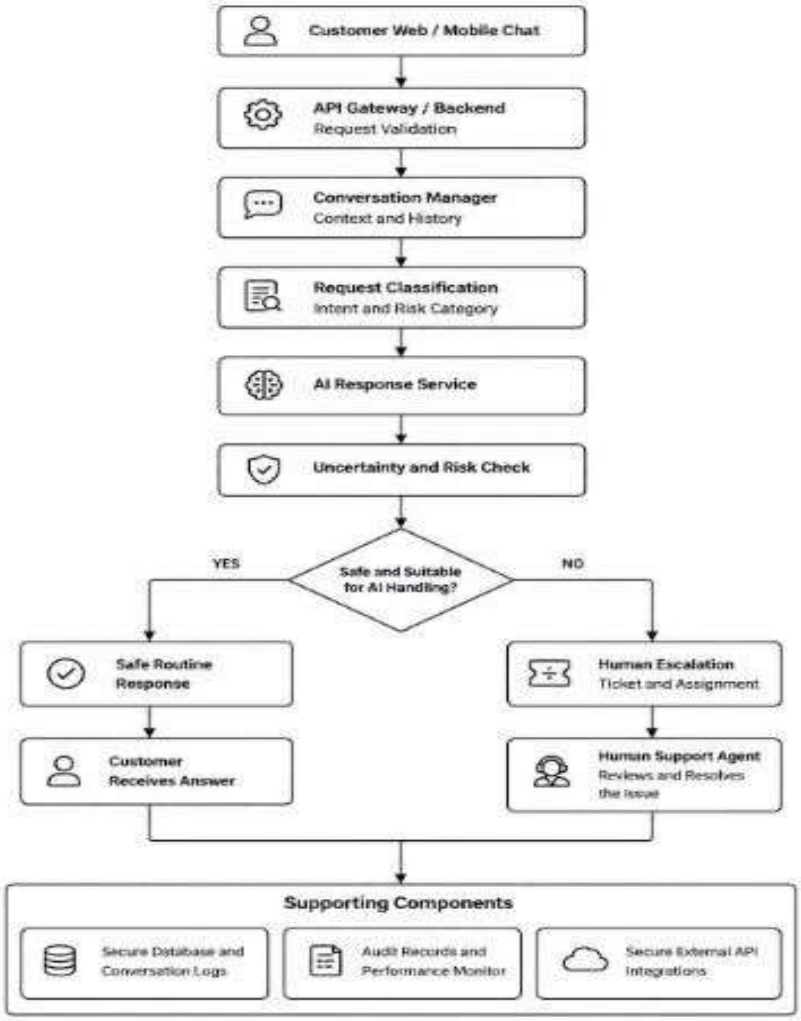
The AI response service generates answers for suitable requests. The uncertainty and risk evaluation component checks whether the request can be handled safely. Based on this evaluation, the system either returns a response or forwards the conversation to the human escalation service.

The escalation service creates or updates a support ticket and provides the human agent with the conversation history and reason for transfer. The data management component stores permitted conversation records and ticket details, while the audit and monitoring component records system events and tracks performance.

External services are accessed through secure APIs with appropriate authentication and authorization. The services can be deployed using containers and scaled independently where needed.



The architecture image should contain the following flow:



12. DETAILED WORKING OF THE SYSTEM

The system works through the following stages.

Step 1: Customer Request Submission

The customer opens the support interface and submits a question or complaint. The platform receives the message along with relevant session information. If the customer is authenticated and the request requires account-specific information, the system verifies the customer's identity before accessing protected data.

Step 2: Request Validation

The backend checks whether the incoming request is properly formatted and meets basic security requirements. Invalid or unauthorized requests are rejected or handled safely. The system should also apply appropriate controls to prevent abuse and excessive requests.

Step 3: Conversation Context Retrieval

The conversation manager retrieves relevant messages from the current interaction. This allows the assistant to understand follow-up questions and avoid asking the customer to repeat information unnecessarily. Only information needed for the current task should be included.

Step 4: Request Classification

The classifier identifies the intent of the message. For example, a customer may ask about an order, a payment, a product, a technical issue, or a complaint. The system uses the category to determine which knowledge source, service, or escalation rule should apply.

Step 5: Information Retrieval and API Integration

When necessary, the platform retrieves approved information from its knowledge base or an authorized external service. For example, an order-status query may require a secure request to the order management system. The platform must verify access permissions before returning private information.

Step 6: AI Response Generation

The AI assistant generates a response using the customer's message, relevant context, and approved information. The response should be understandable, relevant, and consistent with the organization's support policies. The system should avoid presenting unsupported assumptions as confirmed facts.

Step 7: Uncertainty and Risk Evaluation

Before sending the response, the platform checks whether the request is sufficiently clear and whether the response is appropriate for automated handling. The evaluation may consider missing information, uncertainty indicators, request category, policy restrictions, and the potential impact of an incorrect answer.

Step 8: Automated Response or Escalation

If the request is routine and satisfies the system's safety criteria, the response is sent to the customer. If the request is uncertain, sensitive, high-impact, or outside the assistant's permitted scope, the system escalates it to a human agent.

Step 9: Human Review

The human agent receives the customer's message, relevant conversation history, request category, and reason for escalation. The agent reviews the case, asks for additional information if necessary, and provides an appropriate resolution.

Step 10: Logging and Monitoring

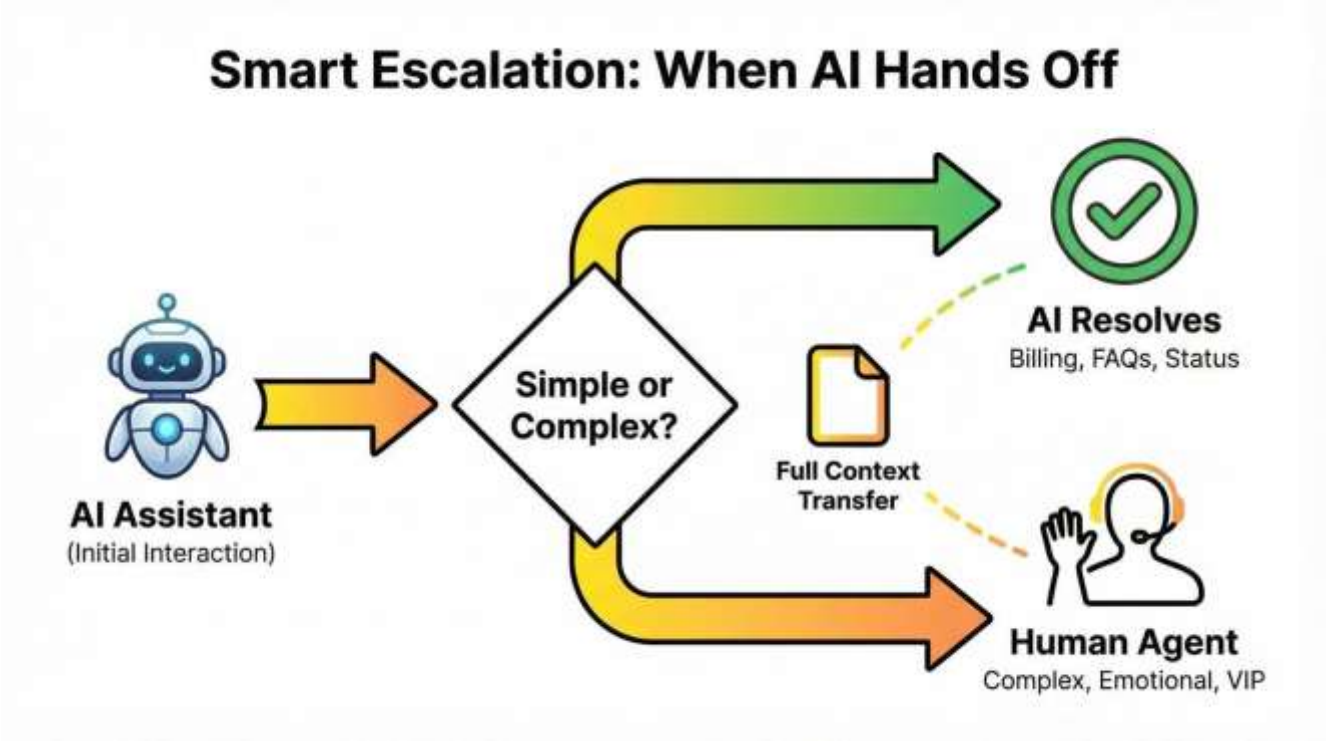
The platform records relevant events, including response generation, escalation, ticket updates, failures, and resolution status. Monitoring tools measure response time, escalation frequency, error rates, availability, and customer feedback. These measurements support future improvement.

13. HUMAN ESCALATION CONTROL FLOWCHART

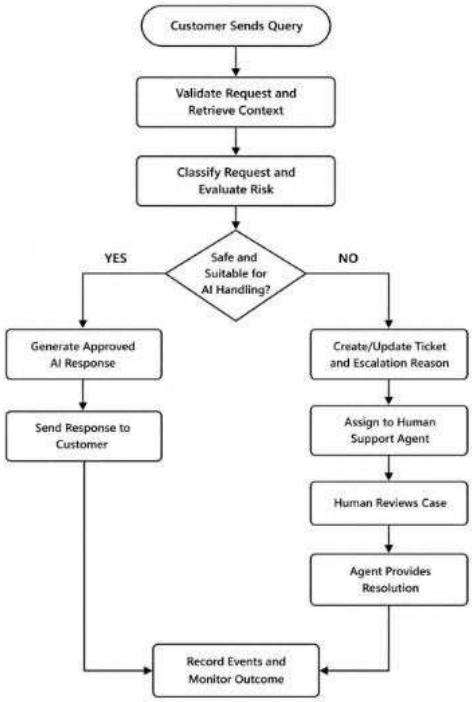
Human escalation control is one of the most important features of the proposed platform. It prevents the AI assistant from handling requests that require additional judgment, verification, or specialist knowledge.

The system evaluates each request using predefined policies and suitable uncertainty or risk indicators. Escalation should not depend only on a single confidence score. Sensitive requests and restricted actions may require human review even when the AI appears confident.

Examples of situations that may require escalation include unresolved complaints, ambiguous requests, suspected fraud, privacy-related concerns, requests involving significant financial consequences, and questions that cannot be answered using approved information.



Use this flowchart for the image:



14. DATA MANAGEMENT AND SECURITY

The platform handles customer messages, conversation history, support tickets, and service activity records. Therefore, data management and security must be included throughout the system design.

Authentication and Authorization: Authentication verifies the identity of a user, while authorization determines which resources the user is allowed to access. Customers should access only their permitted information, and support agents should access cases according to their assigned responsibilities.

Encryption: Sensitive information should be protected during transmission using secure communication protocols such as HTTPS/TLS. Appropriate encryption should also be considered for stored data, depending on the platform's security requirements.

Data Minimization: The system should collect only the information necessary to process a request. Sensitive details should not be included in AI prompts, logs, or API requests unless they are necessary and appropriately authorized.

Secure Audit Logs: The platform should maintain records of significant events such as access attempts, AI responses, escalation decisions, ticket updates, and administrative changes. Logs should be protected against unauthorized access and modification.

Retention and Deletion: Conversation records should be retained only according to the organization's documented retention policy and applicable legal requirements. Where appropriate, personal information should be deleted or anonymized when it is no longer required.

API Security: External API requests should use approved authentication methods, restricted permissions, secure credentials, and appropriate validation. API keys and passwords should not be hard-coded into application source code.

Privacy Protection: Access to personal information should be restricted to authorized purposes. The system should provide suitable procedures for handling privacy requests and investigating possible data exposure.

These controls help protect customer information, improve accountability, and reduce the risk of unauthorized access or data misuse.

15. EXTERNAL API INTEGRATION

External APIs allow the platform to retrieve information and perform approved actions through other organizational systems.

For example, an order management API can provide order status, a customer relationship management system can supply authorized customer details, and a ticketing API can create or update support cases.

When a customer asks about an order, the platform first determines the request type and checks the customer's authorization. It then sends a secure API request to the order service. The response is validated before relevant information is included in the answer.

Similarly, when a conversation requires human support, the escalation service can create a ticket through the ticketing API. The ticket may include the issue category, escalation reason, relevant conversation history, and priority according to approved rules.

API integration should include input validation, authentication, authorization, timeout handling, error handling, and appropriate rate limiting. If an external service is unavailable, the system should provide a safe message or use an approved fallback process rather than inventing information.

This integration enables the platform to provide more useful responses while maintaining control over access to organizational data.

16. TESTING AND REQUIREMENT VALIDATION

Testing is necessary to verify that the platform operates correctly, handles risk appropriately, protects customer data, and meets the stated requirements.

Unit Testing: Individual components, such as request classification, context management, risk evaluation, and ticket creation, are tested separately to verify their expected behavior.

Integration Testing: The interaction between the AI assistant, backend services, database, and external APIs is tested. These tests verify that information is passed correctly and failures are handled safely.

Functional Testing: The system is tested against common customer requests, including product questions, order-status queries, billing questions, complaints, and follow-up messages.

Escalation Testing: The platform is tested using unclear, sensitive, high-impact, and unsupported requests. The purpose is to verify that the appropriate cases are transferred to a human agent.

Security Testing: Tests examine authentication, authorization, access restrictions, input validation, API security, and protection of sensitive information.

Performance Testing: The platform is tested under different workloads to evaluate response time, throughput, resource use, and behavior when many customers interact simultaneously.

Usability Testing: Customers and support agents evaluate whether the interface is understandable and whether escalated cases contain sufficient information to continue the conversation.

Monitoring and Quality Evaluation: Response quality, escalation decisions, unresolved cases, error rates, and customer feedback are reviewed regularly. Test results should be documented and used to improve the platform.

Testing should include both successful and unsuccessful cases. The platform should not be considered ready for deployment until important security, escalation, and reliability requirements have been evaluated.

17. CI/CD, DEPLOYMENT, AND MONITORING

Continuous Integration and Continuous Deployment (CI/CD) practices help developers test and release changes in a controlled and repeatable manner.

When developers modify the source code, the CI pipeline can automatically run code checks, unit tests, integration tests, and security checks. If the required checks pass, a build artifact or container image can be created for further testing.

The updated version should be deployed to a testing or staging environment before production release. After successful validation, the deployment process can release the application to the production environment according to the organization's approval procedures.

Containerization allows application components to run in consistent environments and makes it easier to deploy additional instances when workloads increase. Orchestration tools may be used when the platform requires automated scaling, service recovery, or management of multiple containers.

Monitoring should track application availability, API failures, response time, resource utilization, escalation rates, and other important service indicators. Alerts can notify administrators when error rates increase, external services become unavailable, or performance falls below acceptable levels.

The platform should also support controlled rollback to a previous stable version if a new release causes significant problems. Changes to AI models, prompts, escalation rules, and security settings should be tested and documented before release.

These practices help improve service reliability, maintainability, and operational efficiency.

18. RESPONSIBLE AI PRACTICES

Responsible AI is essential because customer support decisions can affect customer trust, privacy, and access to services.

Human Oversight: The platform must transfer sensitive, high-impact, uncertain, or restricted cases to authorized human staff. Human agents should have sufficient information and authority to review and resolve escalated cases.

Transparency: Customers should be informed when they are interacting with an AI assistant. The system should explain when a request is being transferred to a human agent and, where appropriate, provide the reason.

Fairness: The system should be evaluated to identify whether response quality, escalation behavior, or service outcomes differ unfairly across customer groups or communication styles.

Accuracy and Reliability: AI-generated answers should be based on approved information whenever possible. The system should avoid unsupported claims and provide an appropriate fallback when reliable information is unavailable.

Privacy: Customer information should be handled only for legitimate purposes. Access should be restricted, and sensitive information should be protected in storage, transmission, and logging.

Accountability: The organization should define who is responsible for system configuration, model updates, escalation policies, incident handling, and monitoring.

Continuous Evaluation: AI response quality and escalation performance should be reviewed regularly. Feedback and test results should be used to improve the system without weakening privacy or security protections.

By applying these practices, the platform can provide useful automation while preserving human responsibility and customer confidence.

19. EXPECTED OUTCOMES AND ADVANTAGES

The proposed AI-Assisted Customer Support Platform is expected to provide the following benefits.

Reduced Employee Workload: Routine and repetitive customer requests can be answered automatically, allowing support staff to spend more time on complex and sensitive issues.

Faster Customer Responses: Customers can receive answers to common questions without waiting for a human employee to become available.

Improved Escalation: Uncertain, sensitive, and high-impact requests can be transferred to human agents instead of being handled entirely through automation.

Better Conversation Continuity: Maintaining relevant conversation context reduces repetition and helps human agents understand the customer's issue more quickly.

Improved Security: Authentication, authorization, secure API communication, and protected audit records help reduce risks related to unauthorized access and information exposure.

Scalability: Modular services and containerized deployment allow the system to support increasing workloads by scaling appropriate components.

Better Monitoring: Performance measurements and audit records help organizations identify failures, evaluate response quality, and improve the service over time.

Responsible Automation: Human oversight, privacy protection, transparency, and fairness checks help reduce the risks associated with inappropriate automated responses.

Overall, the system aims to improve operational efficiency while maintaining service quality, customer trust, and appropriate human control.

20. FUTURE ENHANCEMENTS

The platform can be improved in several ways in the future.

1. **Multilingual Support:** Add support for multiple languages so that customers can interact with the assistant in their preferred language.
2. **Voice-Based Assistance:** Integrate speech recognition and text-to-speech technology to support voice-based customer interactions.
3. **Advanced Knowledge Retrieval:** Improve response quality by retrieving relevant information from approved organizational documents and knowledge bases.
4. **Improved Risk Detection:** Develop and evaluate better methods for identifying uncertain requests, sensitive topics, and situations requiring human review.

5. **Intelligent Ticket Prioritization:** Prioritize escalated cases based on predefined business rules, urgency, and the type of issue.
6. **Customer Feedback Analysis:** Analyze customer feedback to identify recurring complaints, common failures, and areas where the support process can be improved.
7. **Advanced Analytics:** Provide reports on response time, resolution rate, escalation frequency, and customer satisfaction to support service planning.
8. **Continuous Model Evaluation:** Evaluate updated models and response policies using controlled tests before they are introduced into the production system.
9. **Improved Availability:** Add stronger recovery mechanisms, redundancy, and deployment strategies to improve reliability during service interruptions.
10. **Integration with More Services:** Connect the platform with additional approved enterprise systems to support a wider range of customer requests.

These enhancements can expand the platform's capabilities while preserving privacy, security, and human oversight.

21. CONCLUSION

The AI-Assisted Customer Support Platform provides a structured approach to automating routine customer interactions while maintaining human control over uncertain, sensitive, and high-impact situations.

The system classifies customer requests, maintains conversation context, generates appropriate AI responses, evaluates uncertainty and risk, and escalates unsuitable cases to human support staff. It also supports secure API integration, protected audit records, performance monitoring, testing, and scalable deployment.

The use of human escalation control helps prevent the AI assistant from handling every request without considering its complexity or potential impact. Security and responsible AI practices further protect customer information and support transparency, fairness, and accountability.

Therefore, the proposed platform can reduce repetitive work, improve response speed, strengthen oversight, and support reliable customer service. By combining AI automation with human judgment, the system provides a practical foundation for safer and more efficient AI-assisted customer support.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25